



Directory > R2 > API >

Workers API > Use R2 from Workers



Copy page



Use R2 from Workers

1. Create a new application with C3

C3 (`create-cloudflare-cli`) is a command-line tool designed to help you set up and deploy Workers & Pages applications to Cloudflare as fast as possible.

To get started, open a terminal window and run:



npm



yarn



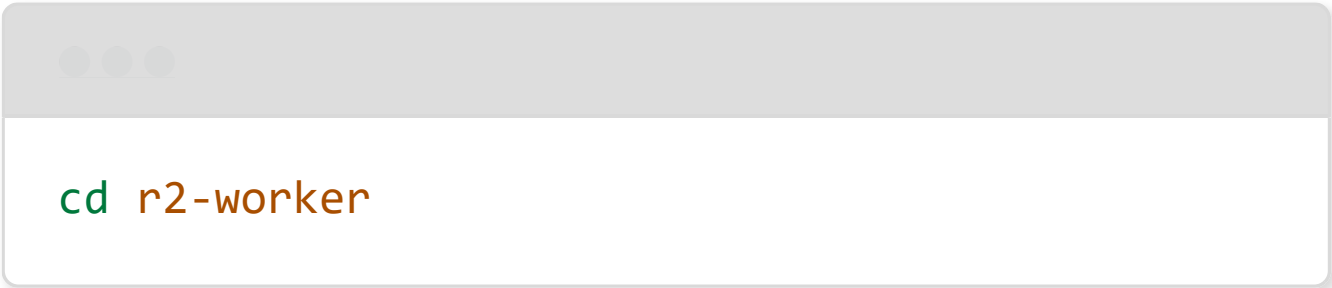
pnpm

```
pnpm create cloudflare@latest r2-worker
```

For setup, select the following options:

- For *What would you like to start with?*, choose Hello World example.
- For *Which template would you like to use?*, choose Worker only.
- For *Which language do you want to use?*, choose JavaScript.
- For *Do you want to use git for version control?*, choose Yes.
- For *Do you want to deploy your application?*, choose No (we will be making some changes before deploying).

Then, move into your newly created directory:



```
cd r2-worker
```

2. Create your bucket

Create your bucket by running:

```
npx wrangler r2 bucket create  
<YOUR_BUCKET_NAME>
```

To check that your bucket was created, run:

```
npx wrangler r2 bucket list
```

After running the `list` command, you will see all bucket names, including the one you have just created.

3. Bind your bucket to a Worker

You will need to bind your bucket to a Worker.

Bindings

A binding is how your Worker interacts with external resources such as [KV Namespaces](#), [Durable Objects](#), or [R2 Buckets](#). A binding is a runtime variable that the Workers runtime provides to your code. You can declare a variable name in your Wrangler file that will be bound to these resources at runtime, and interact with them through this variable. Every binding's variable name and behavior is determined by you when deploying the Worker. Refer to the [Environment Variables](#) documentation for more information.

A binding is defined in the Wrangler file of your Worker project's directory.

To bind your R2 bucket to your Worker, add the following to your Wrangler file. Update the binding property to a valid JavaScript variable identifier and

bucket_name to the <YOUR_BUCKET_NAME> you used to create your bucket in [step 2](#):

{ } wrangler.jsonc



wrangler.toml

```
{
  "r2_buckets": [
    {
      "binding": "MY_BUCKET",
      "bucket_name": "<YOUR_BUCKET_NAME>"
    }
  ]
}
```

For more detailed information on configuring your Worker (for example, if you are using [jurisdictions](#)), refer to the [Wrangler Configuration documentation](#).

4. Access your R2 bucket from your Worker

Within your Worker code, your bucket is now available under the `MY_BUCKET` variable and you can begin interacting with it.

Local Development mode in Wrangler

By default `wrangler dev` runs in local development mode. In this mode, all operations performed by your local worker will operate against local storage on your machine.

If you want the R2 operations that are performed during development to be performed against a real R2 bucket, you can set `"remote" : true` in the R2 binding configuration. Refer to [remote bindings documentation](#) for more information.

An R2 bucket is able to READ, LIST, WRITE, and DELETE objects. You can see an example of all operations below using the Module Worker syntax. Add the following snippet into your project's `index.js` file:

TS TypeScript

JS JavaScript

TypeScript

```
import { WorkerEntrypoint } from
"cloudflare:workers";

export default class extends
WorkerEntrypoint<Env> {
  async fetch(request: Request) {
    const url = new URL(request.url);
    const key = url.pathname.slice(1);

    switch (request.method) {
      case "PUT": {
        await this.env.R2.put(key,
request.body, {
          onlyIf: request.headers,
          httpMetadata: request.headers,
        });
        return new Response(`Put ${key}
successfully!`);
      }
      case "GET": {
        const object = await
this.env.R2.get(key, {
          onlyIf: request.headers,
          range: request.headers,
```

```
    });

    if (object === null) {
        return new Response("Object Not
Found", { status: 404 });
    }

    const headers = new Headers();
    object.writeHttpMetadata(headers);
    headers.set("etag", object.httpEtag);

    // When no body is present,
    preconditions have failed

    return new Response("body" in object ?
object.body : undefined, {
        status: "body" in object ? 200 :
412,

        headers,
    });
}

case "DELETE": {
    await this.env.R2.delete(key);
    return new Response("Deleted!");
}

default:
```

```
    return new Response("Method Not  
    Allowed", {  
        status: 405,  
        headers: {  
            Allow: "PUT, GET, DELETE",  
        },  
    });  
}  
}
```



Prevent potential errors when accessing `request.body`

The body of a [Request](#) ↗ can only be accessed once. If you previously used `request.formData()` in the same request, you may encounter a `TypeError` when attempting to access `request.body`.

To avoid errors, create a clone of the `Request` object with `request.clone()` for each subsequent attempt to access a `Request`'s body. Keep in mind that Workers have a [memory limit of 128 MB per Worker](#) and loading particularly large files into a Worker's memory multiple times may reach this limit. To ensure memory usage does not reach this limit, consider using [Streams](#).

5. Bucket access and privacy

With the above code added to your Worker, every incoming request has the ability to interact with your bucket. This means your bucket is publicly exposed and its contents can be accessed and modified by undesired actors.

You must now define authorization logic to determine who can perform what actions to your bucket. This logic lives within your Worker's code, as it is your application's job to determine user privileges. The following is a short list of resources related to access and authorization practices:

1. [Basic Authentication](#): Shows how to restrict access using the HTTP Basic schema.
2. [Using Custom Headers](#): Allow or deny a request based on a known pre-shared key in a header.

Continuing with your newly created bucket and Worker, you will need to protect all bucket operations.

For PUT and DELETE requests, you will make use of a new `AUTH_KEY_SECRET` environment variable, which you

will define later as a Wrangler secret.

For GET requests, you will ensure that only a specific file can be requested. All of this custom logic occurs inside of an `authorizeRequest` function, with the `hasValidHeader` function handling the custom header logic. If all validation passes, then the operation is allowed.

JavaScript

```
const ALLOW_LIST = ["cat-pic.jpg"];

// Check requests for a pre-shared secret
const isValidHeader = (request, env) => {
  return request.headers.get("X-Custom-Auth-
Key") === env.AUTH_KEY_SECRET;
};

function authorizeRequest(request, env, key) {
  switch (request.method) {
    case "PUT":
    case "DELETE":
      return isValidHeader(request, env);
    case "GET":
      return ALLOW_LIST.includes(key);
    default:
      return false;
  }
}

export default {
  async fetch(request, env, ctx) {
    const url = new URL(request.url);
    const key = url.pathname.slice(1);
```

```
    if (!authorizeRequest(request, env, key))  
    {  
        return new Response("Forbidden", {  
status: 403 });  
    }  
  
    // ...  
},  
};
```

For this to work, you need to create a secret via Wrangler:

```
npx wrangler secret put AUTH_KEY_SECRET
```

This command will prompt you to enter a secret in your terminal:

```
npx wrangler secret put AUTH_KEY_SECRET
```

Enter the secret text you'd like assigned to the variable `AUTH_KEY_SECRET` on the script named `<YOUR_WORKER_NAME>`:



Creating the secret for script name `<YOUR_WORKER_NAME>`



Success! Uploaded secret `AUTH_KEY_SECRET`.

This secret is now available as `AUTH_KEY_SECRET` on the `env` parameter in your Worker.

6. Deploy your Worker

With your Worker and bucket set up, run the `npx wrangler deploy` [command](#) to deploy to Cloudflare's global network:

```
npx wrangler deploy
```

You can verify your authorization logic is working through the following commands, using your deployed Worker endpoint:



Warning

When uploading files to R2 via `curl`, ensure you use `--data-binary`  instead of `--data` or `-d`. Files will otherwise be truncated.



```
# Attempt to write an object without  
providing the "X-Custom-Auth-Key" header  
curl https://your-worker.dev/cat-pic.jpg -X  
PUT --data-binary 'test'
```

```
#=> Forbidden
```

```
# Expected because header was missing
```

```
# Attempt to write an object with the wrong  
"X-Custom-Auth-Key" header value  
curl https://your-worker.dev/cat-pic.jpg -X  
PUT --header "X-Custom-Auth-Key: hotdog" --  
data-binary 'test'
```

```
#=> Forbidden
```

```
# Expected because header value did not match  
the AUTH_KEY_SECRET value
```

```
# Attempt to write an object with the correct  
"X-Custom-Auth-Key" header value
```

```
# Note: Assume that "*****" is the value  
of your AUTH_KEY_SECRET Wrangler secret
```

```
curl https://your-worker.dev/cat-pic.jpg -X  
PUT --header "X-Custom-Auth-Key: *****" --  
data-binary 'test'
```

```
#=> Put cat-pic.jpg successfully!
```

```
# Attempt to read object called "foo"
curl https://your-worker.dev/foo
#=> Forbidden
# Expected because "foo" is not in the
ALLOW_LIST

# Attempt to read an object called "cat-
pic.jpg"
curl https://your-worker.dev/cat-pic.jpg
#=> test
# Note: This is the value that was
successfully PUT above
```

By completing this guide, you have successfully installed Wrangler and deployed your R2 bucket to Cloudflare.

Related resources

1. [Workers Tutorials](#)
2. [Workers Examples](#)

Was this helpful?

[Previous](#)[Next](#)[Presigned](#)[Use the R2 multipart API from](#)[URLs](#)[Workers](#)

Last updated: Sep 30, 2025

Resources

[API](#)[New to Cloudflare?](#)[Directory](#)[Sponsorships](#)[Open Source](#)

Support

[Help Center](#)[System Status](#)[Compliance](#)[GDPR](#)

Company

[cloudflare.com](#)[Our team](#)

Tools

[Cloudflare Radar](#)[Speed Test](#)

Careers

Is BGP Safe Yet?

RPKI Toolkit

Certificate Transparency

Community

 X

 Discord

 YouTube

 GitHub

© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) •  [Cookie Preferences](#)